

Notes about Basic Polyphase Decimation Filters

Jan 25, 2026

- [Introduction](#)
- [The Decimation and Anti-Aliasing FIR Filter Combo](#)
- [Naive Hardware Implementation](#)
- [Reduce number of calculations - Move decimator before multiplier](#)
- [Polyphase decomposition of the original filter](#)
- [The Noble Identity for Decimation](#)
- [Reusing Common Hardware in the Fast Clock Domain](#)
- [Delayed multiplications instead of delayed inputs](#)
- [Conclusion](#)
- [References](#)
- [Footnotes](#)

Introduction

I've been reading up on polyphase filters and multi-rate digital signal processing. It's a broad topic, but as a beginner I need to start with the basics. And to better internalize those, I like to expand the generic math into concretely worked-out, smaller examples.

And if I'm going to write things down anyway, I might as well put them in a blog post, this way I know where to look if I want to review things later.

I hope the content here is useful to someone, but don't assume that I know what I'm doing. There are hundreds of articles on the web on the same topic, so make sure to sample a bunch of them to get different perspectives.

One of the things that clicked with me while writing this, is the benefit of rearranging the mathematical equations so that they reflect the hardware implementation. In the past, I've seen a different of architectures for polyphase filters. I was able to understand them intuitively but linking them to math adds an additional layer of confidence.

So that's one of the things I'm doing here: switch back and forth between math and hardware architecture.

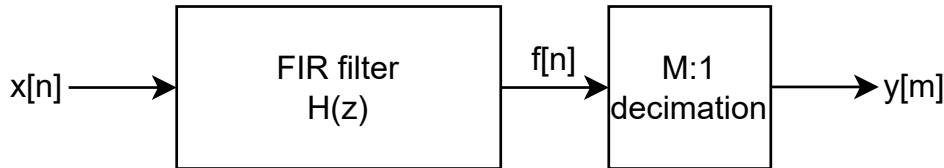
Update: in a later blog post, I added a section about [common DSP notations](#). Check it out first!

The Decimation and Anti-Aliasing FIR Filter Combo

In digital signal processing (DSP), decimation is an operation in which you retain 1 out of every M samples and throw away the rest. It has the benefit of bringing the sample rate down, and thus the amount of data that flows through the system, the clock speed, the number of calculations etc. Decimation is a very common operation.

When following DSP theory, if you want to decimate a signal from a sample rate f_s to a sample rate f_s/M , you first need to apply an anti-aliasing filter that removes all the frequency components above $f_s/(2 \cdot M)$ to make sure that the Nyquist criterium remains valid after the sample frequency has been reduced.¹

When using an FIR filter, the conceptual block diagram looks like this:



Let's do this with 7-tap FIR filter that has transfer function $H(z)$ and a decimation factor M of 3.

$$H(z) = h_0 + h_1 z^{-1} + h_2 z^{-2} + h_3 z^{-3} + h_4 z^{-4} + h_5 z^{-5} + h_6 z^{-6}$$

In this kind of notation, z^{-2} means the input value that was delayed by 2 discrete steps. In electronics, the equation above has a delay line of 6 elements, each element is multiplied by a different value, and the result of those multiplication is added together.

Mathematically, the combination of a filter followed by a decimator is often expressed like this:

$$H(z) \downarrow M$$

Let's now take the following stream of input samples

$$\dots, x[-3], x[-2], x[-1], x[0], x[1], x[2], x[3], x[4], \dots$$

... and apply this stream to the filter equation for multiple time steps:

$$\begin{aligned}
 f[0] &= h_0 x[0] + h_1 x[-1] + h_2 x[-2] + h_3 x[-3] + h_4 x[-4] + h_5 x[-5] + h_6 x[-6] \\
 f[1] &= h_0 x[1] + h_1 x[0] + h_2 x[-1] + h_3 x[-2] + h_4 x[-3] + h_5 x[-4] + h_6 x[-5] \\
 f[2] &= h_0 x[2] + h_1 x[1] + h_2 x[0] + h_3 x[-1] + h_4 x[-2] + h_5 x[-3] + h_6 x[-4] \\
 f[3] &= h_0 x[3] + h_1 x[2] + h_2 x[1] + h_3 x[0] + h_4 x[-1] + h_5 x[-2] + h_6 x[-3] \\
 f[4] &= h_0 x[4] + h_1 x[3] + h_2 x[2] + h_3 x[1] + h_4 x[0] + h_5 x[-1] + h_6 x[-2] \\
 f[5] &= h_0 x[5] + h_1 x[4] + h_2 x[3] + h_3 x[2] + h_4 x[1] + h_5 x[0] + h_6 x[-1] \\
 f[6] &= h_0 x[6] + h_1 x[5] + h_2 x[4] + h_3 x[3] + h_4 x[2] + h_5 x[1] + h_6 x[0] \\
 f[7] &= h_0 x[7] + h_1 x[6] + h_2 x[5] + h_3 x[4] + h_4 x[3] + h_5 x[2] + h_6 x[1] \\
 f[8] &= h_0 x[8] + h_1 x[7] + h_2 x[6] + h_3 x[5] + h_4 x[4] + h_5 x[3] + h_6 x[2] \\
 f[9] &= h_0 x[9] + h_1 x[8] + h_2 x[7] + h_3 x[6] + h_4 x[5] + h_5 x[4] + h_6 x[3]
 \end{aligned}$$

Decimate by selecting 1 out of every 3 filtered sample:

$$y[0] = f[0]$$

$$y[1] = f[3]$$

$$y[2] = f[6]$$

$$y[3] = f[9]$$

Or:

$$y[0] = h_0x[0] + h_1x[-1] + h_2x[-2] + h_3x[-3] + h_4x[-4] + h_5x[-5] + h_6x[-6]$$

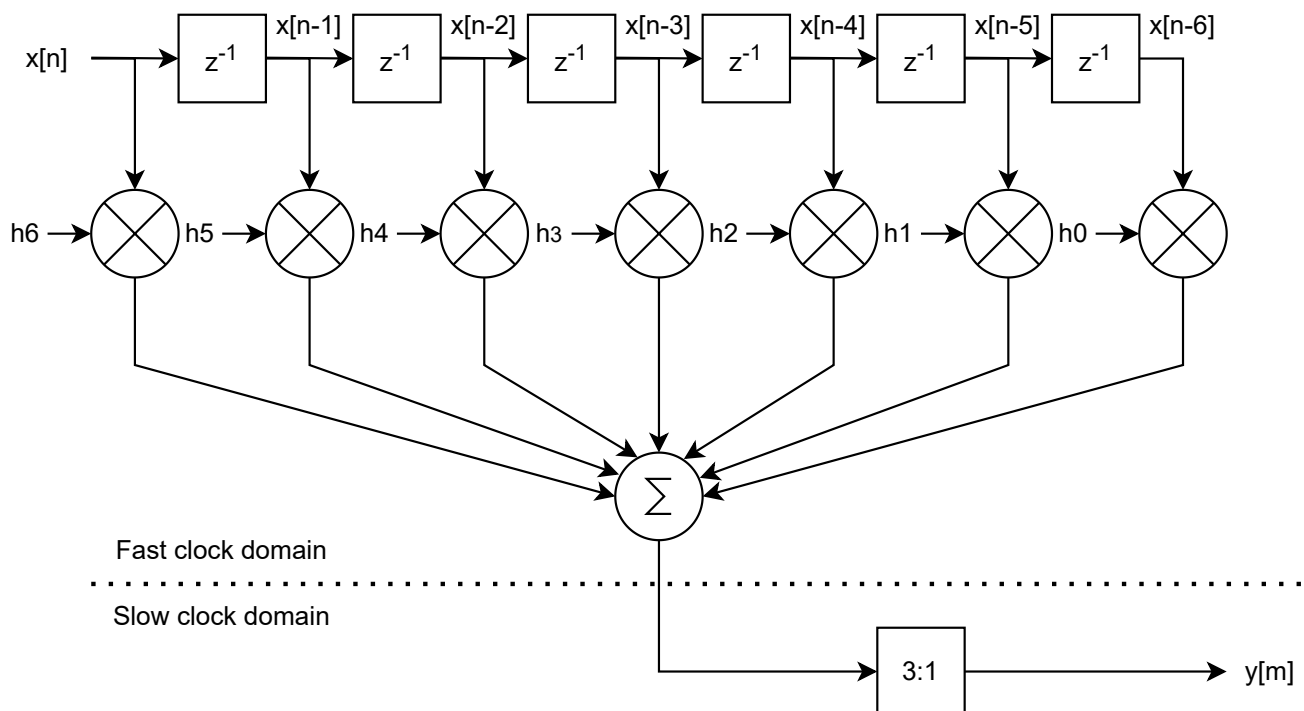
$$y[1] = h_0x[3] + h_1x[2] + h_2x[1] + h_3x[0] + h_4x[-1] + h_5x[-2] + h_6x[-3]$$

$$y[2] = h_0x[6] + h_1x[5] + h_2x[4] + h_3x[3] + h_4x[2] + h_5x[1] + h_6x[0]$$

$$y[3] = h_0x[9] + h_1x[8] + h_2x[7] + h_3x[6] + h_4x[5] + h_5x[4] + h_6x[3]$$

Naive Hardware Implementation

A straight up hardware implementation looks like this:



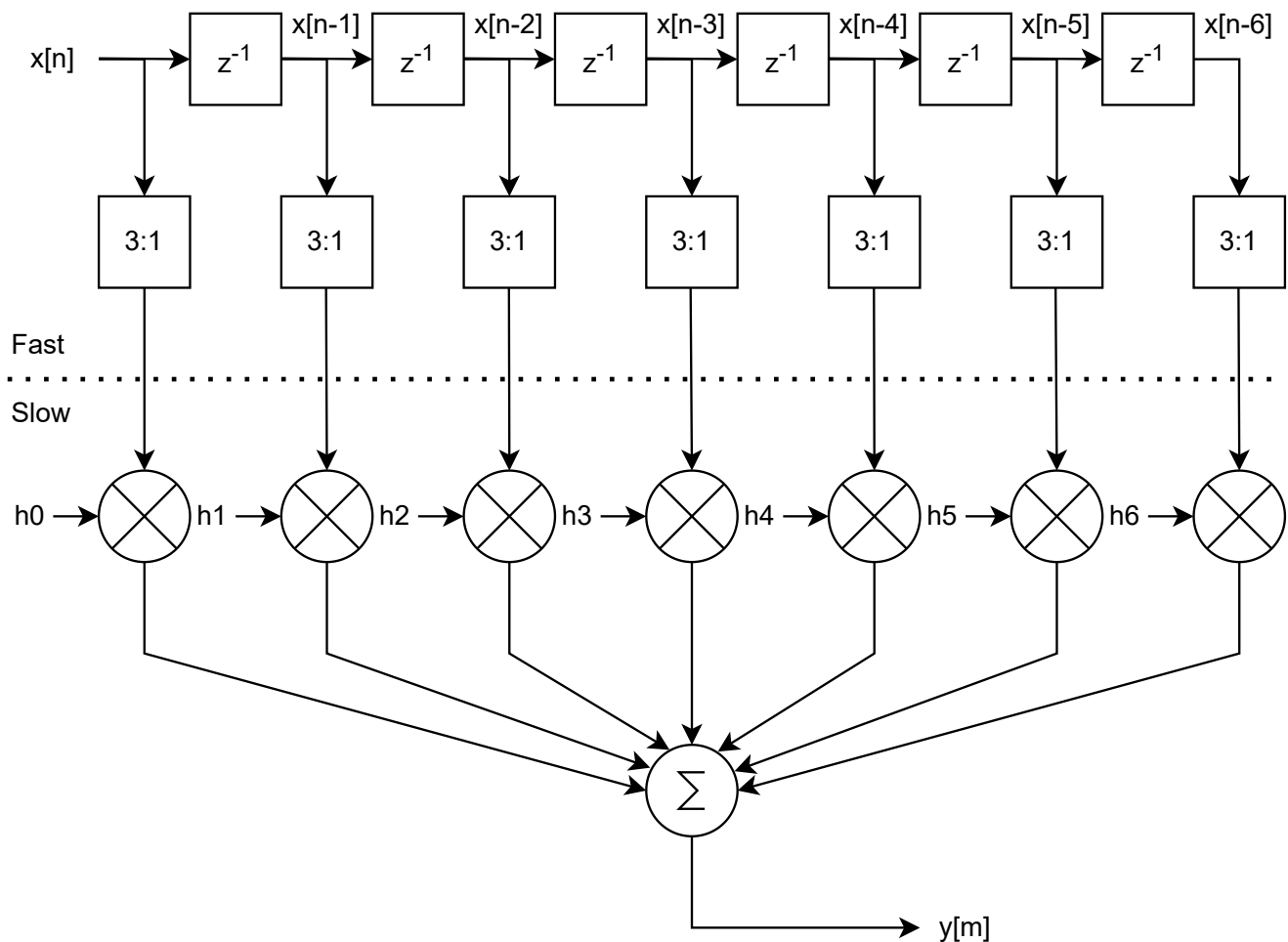
As mentioned before, we have 6 delay elements and 7 multipliers that operate on the each stage of the delay line.

This solution is dumb: we calculate a filter output for every input clock cycle only to throw away 2 out of 3 results. Let's do better.

Reduce number of calculations - Move decimator before multiplier

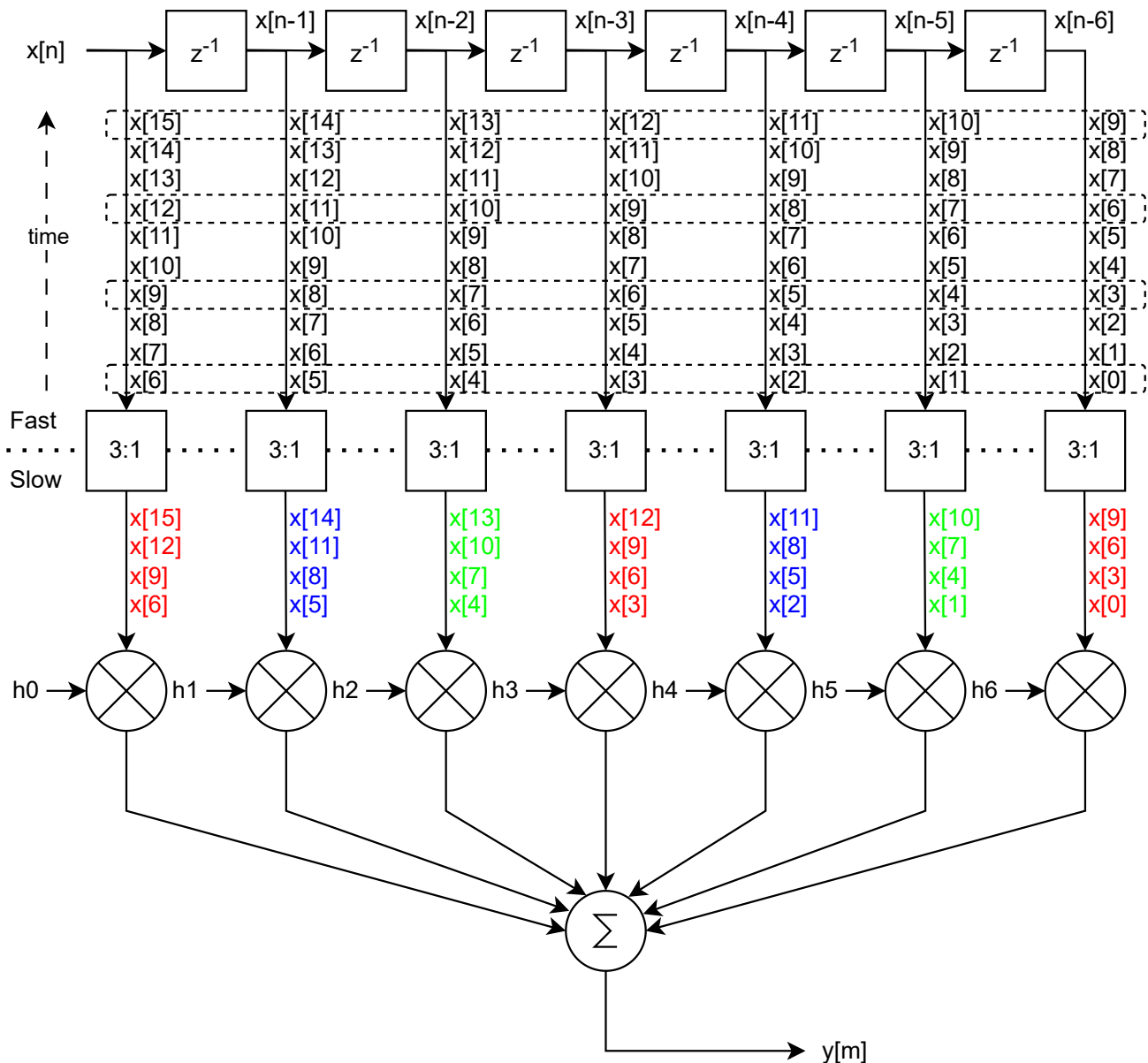
We can reduce the number of calculations by moving the decimator before the filter.

All multiplications still happen at the same time but they can now be performed in a clock domain that is 3 times slower. This definitely reduces power and also reduces the multiplication area in an ASIC process, because timing paths won't be as strict.



While the number of multiplications per unit of time has been reduced by 3, the number of multipliers is still the same.

The data flowing through this architecture looks like this:



When you look at bit closer, you can see that pipes of input samples with the same color have the same data flowing through them: the input feed of the h_0 multiplier sees the same $x[3i]$ samples as the h_3 and the h_6 multipliers, it's just that there is a delay of 1 clock cycle in the slow clock domain for each term. Similarly, h_1 and h_5 multipliers see samples $x[3i + 1]$, and the h_2 and h_6 multipliers see samples $x[3i + 2]$.

Polyphase decomposition of the original filter

Let's take the earlier equation for value $y[3]$ and decorate the 7 terms with the colors of the diagram:

$$y[3] = h_0 x[9] + h_1 x[8] + h_2 x[7] + h_3 x[6] + h_4 x[5] + h_5 x[4] + h_6 x[3]$$

Now split the equation into 3 steps so that each step uses input values $x[i]$ with the same color:

$$\begin{aligned}
 \text{tmp} &= h_0x[9] + h_3x[6] + h_6x[3] \\
 \text{tmp} &= h_1x[8] + h_5x[5] + \text{tmp} \\
 y[2] &= h_2x[7] + h_4x[4] + \text{tmp}
 \end{aligned}$$

What we've done here is split 7-tap filter $H(z)$ into 3 separate sub-filters:

$$H_0(z) = h_0 + h_3z^{-1} + h_6z^{-2}$$

$$H_1(z) = h_1 + h_4z^{-1} + h_7z^{-2}$$

$$H_2(z) = h_2 + h_5z^{-1} + h_8z^{-2}$$

(In our example, h_7 and h_8 are zero.)

The equation of the original filter is now this:

$$H(z) = H_0(z^3) + z^{-1}H_1(z^3) + z^{-2}H_2(z^3)$$

This is the polyphase decomposition of the original filter.

The exponent of 3 in z^3 tells us that input to each sub-filter is a decimated version, because if we substitute z^3 into the set of equations $H_i(z)$, we get:

$$H_0(z^3) = h_0 + h_3z^{3-1} + h_6z^{3-2}$$

$$H_1(z^3) = h_1 + h_4z^{3-1} + h_7z^{3-2}$$

$$H_2(z^3) = h_2 + h_5z^{3-1} + h_8z^{3-2}$$

Or:

$$H_0(z^3) = h_0 + h_3z^{-3} + h_6z^{-6}$$

$$H_1(z^3) = h_1 + h_4z^{-3} + h_7z^{-6}$$

$$H_2(z^3) = h_2 + h_5z^{-3} + h_8z^{-6}$$

The polyphase equation of $H(z)$ is now:

$$H(z) = (h_0 + h_3z^{-3} + h_6z^{-6}) + z^{-1}(h_1 + h_4z^{-3} + h_7z^{-6}) + z^{-2}(h_2 + h_5z^{-3} + h_8z^{-6})$$

Which becomes this:

$$H(z) = (h_0 + h_3z^{-3} + h_6z^{-6}) + (h_1 + h_4z^{-4} + h_7z^{-7}) + (h_2 + h_5z^{-5} + h_8z^{-8})$$

And after reordering the terms and setting h_7 and h_8 to zero, we're back to the definition of $H(z)$ at the start of this blog post:

$$H(z) = h_0 + h_1z^{-1} + h_2z^{-2} + h_3z^{-3} + h_4z^{-4} + h_5z^{-5} + h_6z^{-6}$$

The Noble Identity for Decimation

Those who are studying multi-rate digital signal processing will almost certainly be confronted with the noble identities.

For decimation, the noble identity is formulated as follows²:

$$\downarrow M H(z) \equiv H(z^M) \downarrow M$$

When I first got exposed to that, I thought it was confusing, but after going through the motions of the math equations above, it started to make sense.

What it says is:

Performing a decimation and applying those samples to filter $H(z)$ is equivalent to applying the same filter to every M -th sample and then doing the decimation.

Let's look back at the polyphase decomposition of our original $H(z)$:

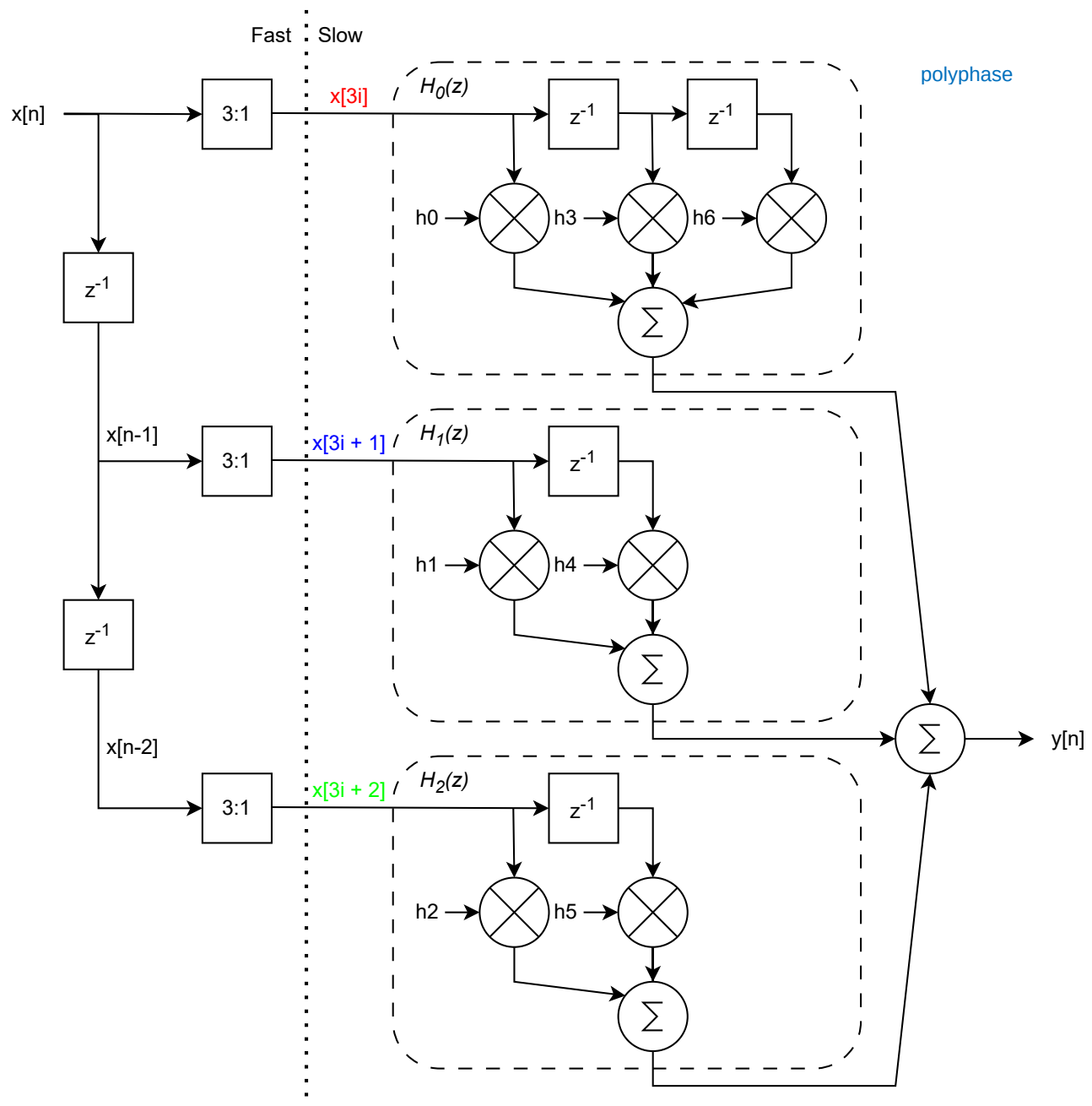
$$H(z) = H_0(z^3) + z^{-1}H_1(z^3) + z^{-2}H_2(z^3)$$

It important to note that we can't apply the noble identity to our $H(z)$ directly, because its coefficients h_1, h_2, h_4 and h_5 are non-zero. But we *can* apply it to the 3 individual phases.

Like this:

$$H(z) \downarrow 3 = (\downarrow 3 H_0(z)) + z^{-1}(\downarrow 3 H_1(z)) + z^{-2}(\downarrow 3 H_2(z))$$

Converted to a hardware diagram:



It's not immediately obvious, but this last diagram is similar to the previous one after we've rearranged some items:

- there's now 1 decimator per phase instead of one per coefficient.
- a single bank of 7 multipliers and one addition has been refactored into 3 banks of multipliers with addition, and then one final addition.
- each multiplier-addition bank has its own delay elements.

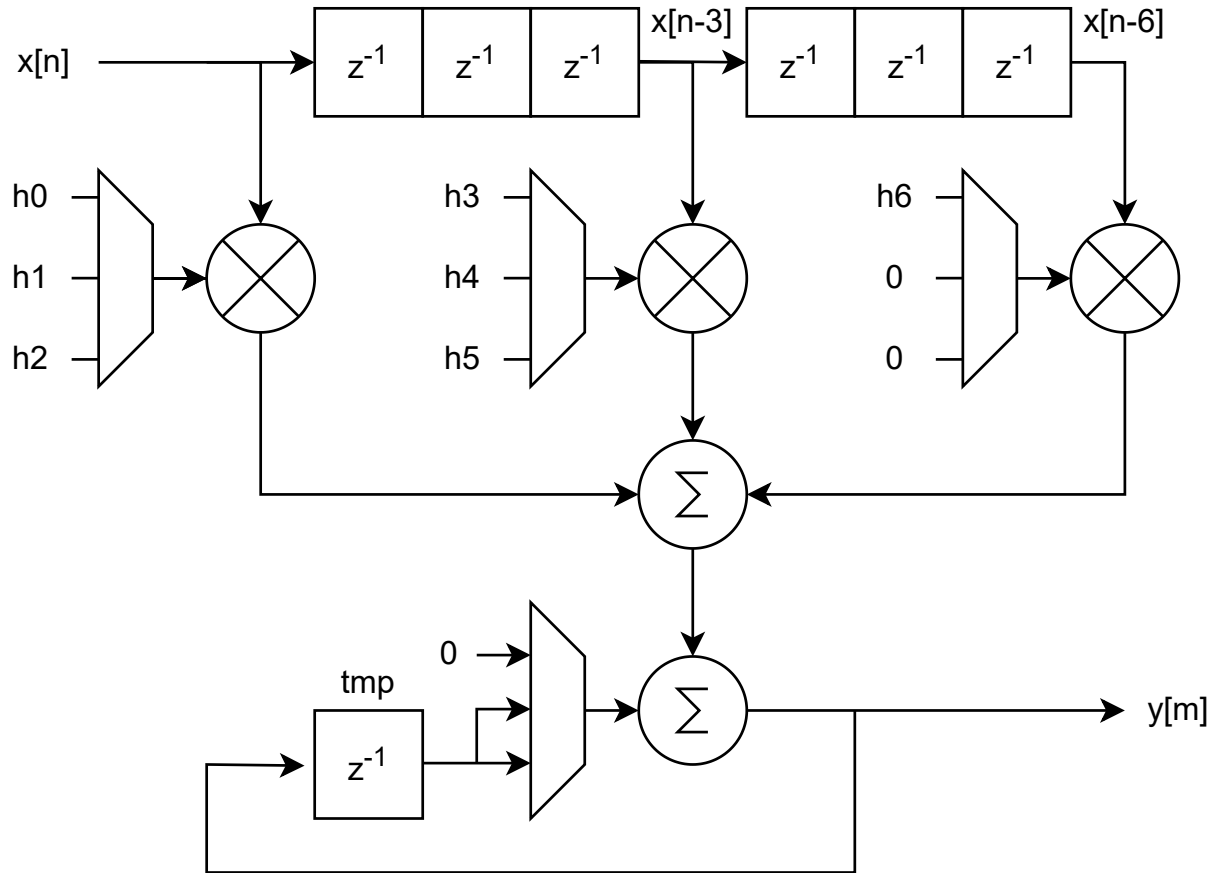
Reusing Common Hardware in the Fast Clock Domain

In the previous diagram, it's clear that there's a lot of common hardware between the different phases. We can exploit that by doing everything in the fast clock domain and reuse the hardware that's used for one phase for the other phases.

Recall the previous equation where the result was calculated in 3 steps:

$$\begin{aligned} \text{tmp} &= h_0x[9] + h_3x[6] + h_6x[3] \\ \text{tmp} &= h_1x[8] + h_5x[5] + \text{tmp} \\ y[2] &= h_2x[7] + h_4x[4] + \text{tmp} \end{aligned}$$

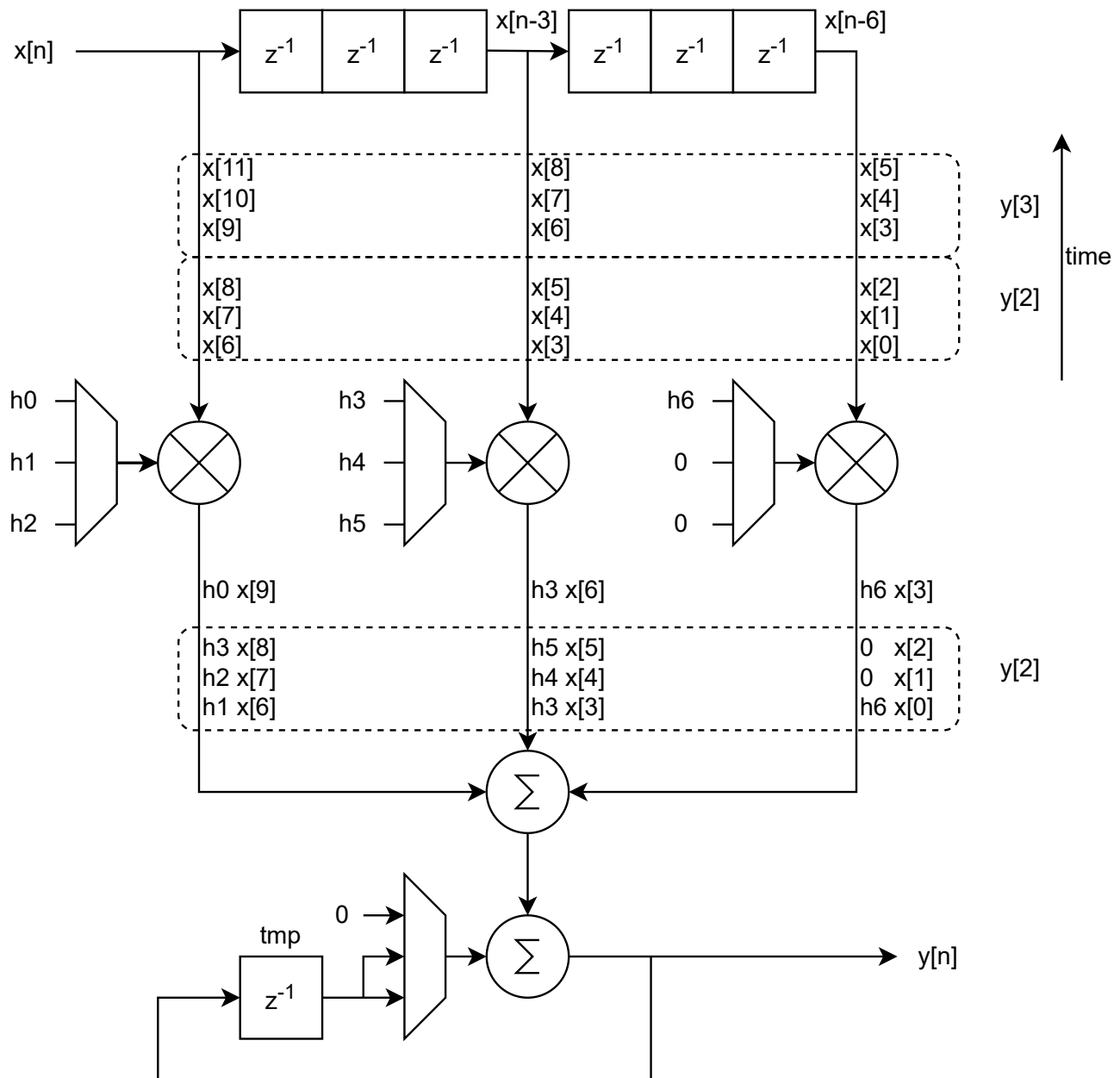
Now check out this diagram:



Everything happens in the fast clock domain, but there are only 3 multipliers instead of 7 and we're only adding 4 numbers together at any time. The only extra cost is a register to store the *tmp* value, and each of the multipliers has a multiplexer to rotate between different coefficients.

There is only one $y[m]$ output every 3 clock cycles.

Here's the same diagram annotated with intermediates values for different time steps:



Delayed multiplications instead of delayed inputs

In the previous diagram, the inputs are delayed and the multiplications summed together. But that's not the only way to implement this.

Let's start again from the original equation:

$$H(z) = h_0 z^0 + h_1 z^{-1} + h_2 z^{-2} + h_3 z^{-3} + h_4 z^{-4} + h_5 z^{-5} + h_6 z^{-6}$$

Reformat:

$$\begin{aligned} H(z) = & (h_0 z^0 + h_1 z^{-1} + h_2 z^{-2}) \\ & + (h_3 z^{-3} + h_4 z^{-4} + h_5 z^{-5}) \\ & + (h_6 z^{-6}) \end{aligned}$$

Extract common z^{-3} and z^{-6} :

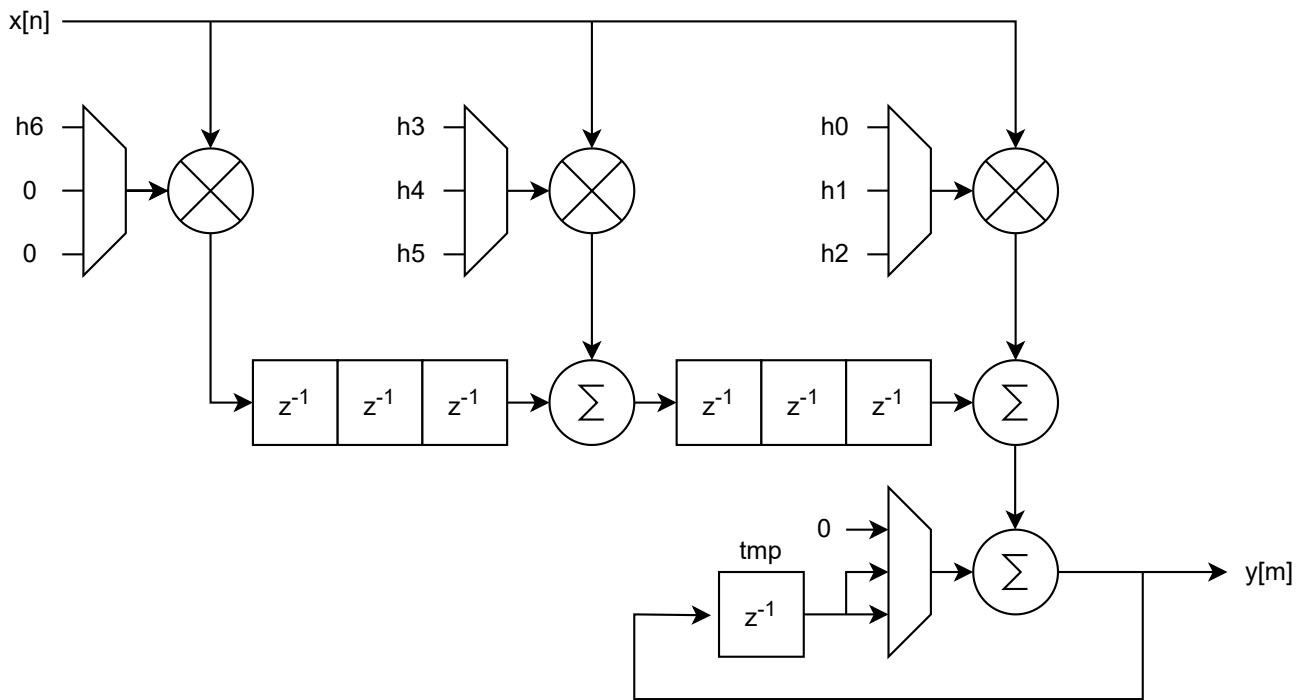
$$\begin{aligned}
 H(z) = & (h_0 z^0 + h_1 z^{-1} + h_2 z^{-2}) \\
 & + z^{-3} (h_3 z^0 + h_4 z^{-1} + h_5 z^{-2}) \\
 & + z^{-6} (h_6 z^0)
 \end{aligned}$$

Extract common z^{-3} :

$$\begin{aligned}
 H(z) = & (h_0 z^0 + h_1 z^{-1} + h_2 z^{-2}) \\
 & + z^{-3} [(h_3 z^0 + h_4 z^{-1} + h_5 z^{-2}) \\
 & + z^{-3} (h_6 z^0)]
 \end{aligned}$$

We now have a nested structure, with a delay of 3 for each nesting level.

In hardware that looks like this:



This structure is not intrinsically worse or better than the previous one, the architecture to use will depend on the technology that you're mapping it to. On FPGAs, for example, you should choose something that makes efficient use of the built-in pipelining registers inside their DSP blocks.

Conclusion

This only scratches the surface of polyphase filters. I didn't even mention interpolation, which does the opposite of decimation but has very similar computational characteristics. I plan to cover addition of topics in the future, especially the expansion of a polyphase filter into a polyphase filter bank. That said, I can't promise a timeline.

References

- [Stackexchange - How to implement Polyphase filter?](#)

Making a polyphase filter implementation is quite easy; given the desired coefficients for a simple FIR filter, you distribute those same coefficients in "row to column" format into the separate polyphase FIR components

Other blog posts in this series:

- [Complex Heterodynes Explained](#)
- [The Stunning Efficiency and Beauty of the Polyphase Channelizer](#)
- [Polyphase Channelizers with Frequency Offset - a Bluetooth LE Example](#)

Footnotes

1. This is not entirely true. You can also apply a bandpass anti-aliasing filter that only retains a part of the spectrum above the new sample rate, and use decimation to bring that section down to the baseband. But that's a topic for a future blog post. [↗](#)
2. $\equiv$ means "is equivalent to." [↗](#)

Electronics etc...

Electronics etc...

 [tomverbeure](#)

 [tom_verbeure](#)